

AIAC Assignment - 8.5

Name: K.Chaitanya

Ht.no: 2303A51677

Batch no:22

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.
- Requirements:
 - Username length must be between 5 and 15 characters.
 - Must contain only alphabets and digits.
 - Must not start with a digit.
 - No spaces

allowed. Example Assert

Test Cases:

```
assert is_valid_username("User123") ==
```

True assert

```
is_valid_username("12User") == False
```

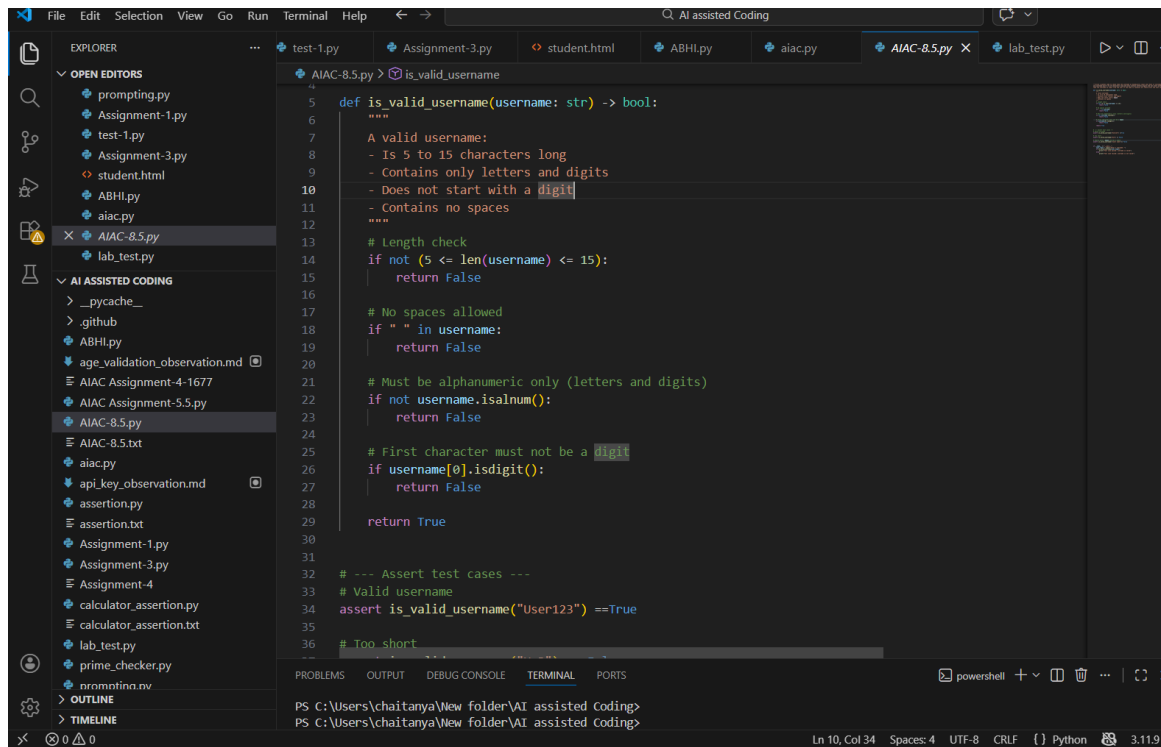
```
assert is_valid_username("Us er") ==
```

False Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

Prompt:

write a python code to check take the username as dynamically input and check if username is valid or not and generate 3 assert test cases and username must 5 to 15 characters long and only contain letters,digits and not start with digit, no spaces allowed take the username as input from the user if the name is contains all these print the test case passed otherwise print failed



```

32 # --- Assert test cases ---
33 # Valid username
34 assert is_valid_username("User123") == True
35
36 # Too short
37 assert is_valid_username("Us3") == False
38
39 # Starts with a digit and has a space
40 assert is_valid_username("1user name") == False
41
42
43 if __name__ == "__main__":
44     user_input = input("Enter a username: ")
45     if is_valid_username(user_input):
46         print("Test case passed: username is valid")
47     else:
48         print("Test case failed: username is not valid")
49
50

```

Output:

```

Enter a username: chaitanya
Test case passed: username is valid
PS C:\Users\chaitanya\New folder\AI assisted Coding>

```

- **Task:** Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.
- **Requirements:**
 - If input is an integer, classify as "Even" or "Odd".
 - If input is 0, return "Zero".
 - If input is non-numeric, return "Invalid"

Input". Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- **Function correctly classifying values and passing all test cases.**

Prompt: Take an integer as dynamic input from the user and check whether the number is even or odd. Generate 3 assert test cases to validate the logic. If the condition is satisfied print "Test Case Passed", otherwise print "Test Case Failed".

```
'''Take an integer as dynamic input from the user and check whether the number is even or odd.
Generate 3 assert test cases to validate the logic.
If the condition is satisfied print ***Test Case Passed***, otherwise print ***Test Case Failed***.
...'''

def is_even_or_odd(num: int) -> str:
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

# --- Assert test cases ---
assert is_even_or_odd(4) == "Even"
assert is_even_or_odd(7) == "Odd"
assert is_even_or_odd(0) == "Even"

if __name__ == "__main__":
    try:
        user_input = int(input("Enter an integer: "))
        result = is_even_or_odd(user_input)
        print(f"The number {user_input} is {result}.")
    except ValueError:
        print("Invalid input. Please enter an integer.")
```

OUTPUT:

```
auncher' '54910' '--' 'C:\Users\chaitanya\New folder\AI assisted Coding\AI
Enter an integer: 12
The number 12 is Even.
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

Task #3 (Palindrome Checker – Apply AI for String Normalization)

- **Task:** Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.
- **Requirements:**
 - Ignore case, spaces, and punctuation.
 - Handle edge cases such as empty strings and single

characters. Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") == True
```

```
assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

Prompt:

Write a Python function `is_palindrome()` that checks whether a given string is a palindrome or not, ignoring spaces and case sensitivity.

Generate 3 assert test cases such as `is_palindrome("Madam") == True`, `is_palindrome("A man a plan a canal Panama") == True`, and `is_palindrome("Python") == False`.

```
'''
Write a Python function `is_palindrome()` that checks whether a given string is a palindrome or not, ignoring spaces and case sensitivity.
Generate 3 assert test cases such as `is_palindrome("Madam") == True`, `is_palindrome("A man a plan a canal Panama") == True`, and `is_palindrome("Python") == False`.
The function should correctly identify palindromes and pass all AI-generated tests.
'''
def is_palindrome(s: str) -> bool:
    # Remove spaces and convert to lowercase
    cleaned_string = s.replace(" ", "").lower()

    # Check if the cleaned string is equal to its reverse
    return cleaned_string == cleaned_string[::-1]

# --- Assert test cases ---
assert is_palindrome("Madam") == True
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("Python") == False
if __name__ == "__main__":
    user_input = input("Enter a string: ")
    if is_palindrome(user_input):
        print("The string is a palindrome.")
    else:
        print("The string is not a palindrome.")
```

OUTPUT:

```
PS C:\Users\chaitanya\New folder\AI assisted Coding> c::; cd 'c:\Users\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\chaitanya\.vscode\launcher' '63359' '--' 'C:\Users\chaitanya\New folder\AI assisted Coding'
Enter a string: madam
The string is a palindrome.
PS C:\Users\chaitanya\New folder\AI assisted Coding> 
```

Task Description

4) (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.
- Methods:
 - deposit(amount)
 - withdraw(amount)
 - get_balance()

Example Assert Test Cases:

```
acc = BankAccount(1000)
```

```
acc.deposit(500)
```

```
assert acc.get_balance() == 1500
```

```
acc.withdraw(300)
```

```
assert acc.get_balance() == 1200
```

Expected Output #4:

Fully functional class that passes all AI-generated assertions

Prompt:

Write a Python program that first generates at least 3 assert-based test cases for a BankAccount class and then implements the class.

The class should include the methods deposit(amount), withdraw(amount), and get_balance() to manage account transactions.

The final implementation should be a fully functional BankAccount class that successfully passes all AI-generated assertions.

```

94
95 '''write a Python program that first generates at least 3 assert-based test cases for a BankAccount class and then implements the class.
96 The class should include the methods deposit(amount), withdraw(amount), and get_balance() to manage account transactions.
97 The final implementation should be a fully functional BankAccount class that successfully passes all AI-generated assertions.'''
98
99 class BankAccount:
100     def __init__(self):
101         self.balance = 0
102     def deposit(self, amount: float):
103         if amount > 0:
104             self.balance += amount
105     def withdraw(self, amount: float):
106         if 0 < amount <= self.balance:
107             self.balance -= amount
108     def get_balance(self) -> float:
109         return self.balance
110
111 # --- Assert test cases ---
112 account = BankAccount()
113 account.deposit(100)
114 assert account.get_balance() == 100
115 account.withdraw(30)
116 assert account.get_balance() == 70
117 account.withdraw(100)
118 assert account.get_balance() == 70
119
120 if __name__ == '__main__':
121     account = BankAccount()
122     account.deposit(200)
123     print(f"Balance after deposit: {account.get_balance()}")
124     account.withdraw(50)

```

OUTPUT:

```

launcher '61103' '--' 'C:\Users\chaitanya\New folder\AI
Balance after deposit: 200
Balance after withdrawal: 150
Balance after attempted over-withdrawal: 150
All test cases passed successfully.

```

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

- **Task:** Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.
- **Requirements:**
 - Must contain @ and .
 - Must not start or end with special characters.
 - Should handle invalid formats

gracefully. Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

Prompt:

Write a Python function `validate_email(email)` that validates whether a given email ID is valid or not.

Generate at least 3 assert test cases to verify the function.

The email must contain @ and . must not start or end with special characters, and should handle invalid formats gracefully while passing all AI-generated test cases.

```

128 """Write a Python function validate_email(email) that validates whether a given email ID is valid or not.
129 Generate at least 3 assert test cases to verify the function.
130 The email must contain @ and ., must not start or end with special characters, and should handle invalid formats gracefully while passing all AI-generated test cases."""
131 def validate_email(email: str) -> bool:
132     # Check if email is a string and not empty
133     if not isinstance(email, str) or not email:
134         return False
135
136     # Check if email starts or ends with a special character
137     special_chars = "!@#$%^&*()~.-"
138     if email[0] in special_chars or email[-1] in special_chars:
139         return False
140
141     # Check for presence of @ and .
142     if "@" not in email or "." not in email:
143         return False
144
145     # Split by @ and check for valid local part and domain part
146     parts = email.split("@")
147     if len(parts) != 2:
148         return False
149
150     local_part, domain_part = parts
151
152     # Ensure both parts are non-empty
153     if not local_part or not domain_part:
154         return False
155
156     # Ensure domain part contains at least one dot
157     if "." not in domain_part:
158         return False
159
160     # Ensure no consecutive dots in local part
161     if "." in local_part:
162         return False
163
164     return True

```

OUTPUT:

```
validate_email('user@example.com') returned True, expected True - PASSED  
validate_email('userexample.com') returned False, expected False - PASSED  
validate_email('@gmail.com') returned False, expected False - PASSED  
validate_email('test@domain.co.uk') returned True, expected True - PASSED  
validate_email('invalid.email') returned False, expected False - PASSED  
validate_email('.invalid@example.com') returned False, expected False - PASSED  
validate_email('valid.email@domain.org') returned True, expected True - PASSED
```